On the non-uniform semantics of *return-type-requirement*s

Document number:	P1452R2
Date:	2019-07-18
Project:	ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup:	Core, LWG
Revises:	P1452R1

Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Changelog

Changes from R1

- Added EWG discussion result.
- Added wording implementing the direction given at EWG for review by CWG and LWG.
- Revised the paper as requested by CWG and LWG.

Changes from R0

- Updated based on presentation to EWGI at the 2019 Kona meeting.
- Added comparative counts of *return-type-requirement*s from N4800 in terms of `Same` versus `ConvertibleTo`, etc.

Introduction

What we have as a *return-type-requirement* today is a misnomer in both the syntactic and semantic sense. The Concepts TS introduced two kinds of semantic constraints that syntactically took the form of a *trailing-return-type*: the deduction constraint and the implicit conversion constraint. In the time since, deduction constraints were first changed by the lack of constrained placeholders until the adoption of P1141R1 at the 2018 San Diego meeting (becoming syntactically different from a *trailing-return-type*), and then—with P1084R2—they ceased to deduce in the manner associated with return type deduction. The syntactic space occupied by deduction constraints in the Concepts TS is now occupied by something else altogether; however, the implicit conversion constraint lives on. Yet the two are both *return-type-requirement*s, using the `->` token.

The inconsistency between the uses of the `->` token in the same context seems unfortunate; and the *trailing-return-type* form of *return-type-requirement*, with its current semantics, appears to be easily replaceable. Its presence in the language seems to be an unnecessary complication that may frustrate future extensions.

A possible future: Generalized placeholder deduction

One aspect of the Concepts TS that did not make the cut into C++2a is “generalized `auto`”. In particular, something like this:

```
void f(std::unique_ptr<Concept auto> p); //
    //                 ^^^ Constraint applied to the pointee type.
```

would mean the same as:

```
template <Concept T> void f(std::unique_ptr<T> p);
```

Let us then consider the application of “generalized `auto`” to *return-type-requirement*s where an analogue would look like this:

```
requires {
  { f() } -> std::unique_ptr<Concept auto>; //
      // Constrain the pointee type.
}
```

and ask ourselves what the following should mean:

```
{ x } -> Concept auto;
```

`-> Type` versus `-> Concept`

The `-> Type` form of *return-type-requirement* behaves consistently with the behaviour of `return E;` for a function that is declared to return *Type* except that placeholder types are allowed for *trailing-return-type*s in function declarations and not for *return-type-requirement*s. That is, the semantics of the various *compound-requirement*s in

```
requires { { E } -> Type; };
requires (void (&f)(Type)) { f(E); };
requires { [](Type) {}(E) }; };
requires { { E } -> ConvertibleTo<Type>; };
```

are roughly the same, with the first two being equivalent. The last form is less aware of the context in terms of access checking, null pointer conversion, and other cases where perfect forwarding is less-than-perfect. It also requires explicit conversion to be possible, and not only implicit conversion. Both the first and the last forms are used by the library in N4800, the pre-San Diego working draft (with five instances of the first form and nine of the last form). Note, however, that `-> Same<`*Type*`>` is more common at forty instances.

In contrast to `-> Type`, the `-> Concept` form of *return-type-requirement* most notably does not use a type after the `->` token, but is instead one of the places where a concept name has a special meaning. It also deduces in a manner different from either of *Concept* `auto` or *Concept* `decltype(auto)` would for a function whose declared return type contains such a placeholder type, and it does not involve a check for convertibility.

`-> Concept auto` is (syntactically) `-> Type`

We now come back to the question from earlier:

```
{ x } -> Concept auto; // What should this mean?
```

Maybe (with `decltype((x))` being `int &`):

```
{ x } -> Concept; // Concept<int &> is satisfied.
```

or, maybe:

```
auto f() -> Concept auto { return x; } // Concept<int>; no "&" (!)
```

Extending the current `-> Type` case to allow placeholder types would leave us with a problem where the extension of the current semantics would lead to a difference in the meaning of the first two *compound-requirement*s in

```
requires {
  { E } -> Concept;
  { E } -> Concept auto;
  { [](Concept auto) {}(E) };
};
```

However, we note that the third *compound-requirement* already expresses a deduction constraint in the TS style (except that the TS wording causes access checking, etc. to be ignored). It would seem that extending from the current semantics of `-> Type` *return-type-requirement*s is not very profitable.

Concept `auto` with a different precedent

Given:

```
void g(Concept auto);
```

Deduction does not only happen through deducing template arguments from a function call (akin to the current `-> Type` deduction). It may also happen through deducing template arguments from a function declaration, e.g.,

```
class A {
  static int x;
  friend void ::g(decltype((x))); // Would deduce int &.
};
```

This latter form of deduction (deducing from a type as opposed to deducing from an expression) is a possible interpretation of the change to `-> Concept` adopted through P1084. This can also be seen as the difference between requiring convertibility and requiring sameness. Through these characterizations, we can make the case that `-> Type` does not have to retain the same semantics as return type deduction.

Possibility for a more powerful `-> Type` (recap)

Today's placeholder types are limited in context; however, the Concepts TS allowed deduction constraints like

```
-> std::vector<Boolean>
```

through "generalized `auto`".

We note that this sort of "type pattern" is expressed in the language as a type. Thus, if we retain `-> Type` with its current semantics, we will not gain the benefit of applying P1084 to such cases if we were to adopt them into the language. That `-> Concept` and `-> Concept auto` would have different semantics would be highly unfortunate, as will having `-> Type` behave rather differently depending on whether a placeholder is present or not.

We also note that `Concept auto` already has the ability to take on semantics that behave more like `Concept` in `-> Concept`. That is, we are free (at this time) to make it such that `-> Type` has semantics matching that of `-> Same<Type>`, by far the most commonly used semantic in N4800, by applying deduction from a type as opposed to deduction from an expression. Benefits would be that `-> Concept auto` would work like

`-> Concept` does in N4800, and that replacing a concrete type in a *return-type-requirement* with a concept that the type models would not cause the *compound-requirement* to reject previously accepted cases.

Proposal for C++20: remove `-> Type`

The `-> Type` form of *return-type-requirement* is underpowered and does not bring additional expressiveness to the language. It is on the wrong side of the split between `ConvertibleTo` and `Same` as used with *compound-requirement*s in N4800, and it introduces complications for future extensions; therefore, it is proposed that the *trailing-return-type* form of *return-type-requirement* be removed. At the same time, it is proposed that *trailing-type-requirement* is a more appropriate name.

EWG discussion result

The paper was presented at the 2019 Cologne meeting. Certain members of EWG indicated that the `Same<Type>` semantics could be adopted as an alternative to removing the *trailing-return-type* form of *return-type-requirement*. Two polls were taken, the first to perform the removal as proposed by the paper presented, and the second to adopt the semantics of `Same`. The first poll gained consensus, and had more consensus than the second poll.

CWG discussion result

CWG considered the whether a feature-test macro should be added or modified based on this paper. The consensus of CWG was that the primary value of a feature test macro would come from the ability to write code that can benefit from using the feature conditionally and can reasonably maintain similar semantics in the alternative. In the case of the feature in question, using the feature conditionally is disadvantageous in comparison to simply using an alternative.

Proposed wording

Changes are relative to N4820.

In subclause 7.5.7 [expr.prim.req], modify the example in paragraph 3:

> [*Example*: A common use of *require-expression*s is to define requirements in concepts such as the one below:
>
> ```
> template<typename T>
> concept R = requires (T i) {
>     typename T::type;
>     {*i} -> ConvertibleTo<const typename T::type&>;
>   };
> ```
>
> … —*end example*]

In subclause 7.5.7.3 [expr.prim.req.compound], remove from the grammar:

> *return-type-requirement*:

~~*trailing-return-type*~~
-> *type-constraint*

In subclause 7.5.7.3 [expr.prim.req.compound], modify in paragraph 1:

- If the *return-type-requirement* is present, then:
  - Substitution of template arguments (if any) into the *return-type-requirement* is performed.
  - ~~If the *return-type-requirement* is a *trailing-return-type* ([dcl.decl]), `E` is implicitly convertible to the type named by the *trailing-return-type*. If conversion fails, the enclosing *requires-expression* is `false`.~~
  - ~~If the *return-type-requirement* is of the form `->` *type-constraint*, then the~~ The contextually-determined type being constrained by the *type-constraint* is `decltype((E))`. The immediately-declared constraint ([temp]) of `decltype((E))` shall be satisfied. [*Example*: … *—end example*]

In subclause 7.5.7.3 [expr.prim.req.compound], modify the example in paragraph 2:

[*Example*: …

```
template<typename T> concept C2 = requires(T x) {
  {*x} -> Same<typename T::inner>;
};
```

The *compound-requirement* in `C2` requires that `*x` is a valid expression, that `typename T::inner` is a valid type, and that ~~`*x` is implicitly convertible to `typename T::inner`~~ `Same<decltype((*x)), typename T::inner>` is satisfied. … *—end example*]

In subclause 13.6.8 [temp.concept], modify the example in paragraph 2:

A *concept-definition* declares a concept. Its identifier becomes a *concept-name* referring to that concept within its scope. [*Example*:

```
template<typename T>
concept C = requires(T x) {
  { x == x } -> ConvertibleTo<bool>;
};
```

… *—end example*]

In subclause 23.3.2.3 [iterator.traits], modify in paragraph 2:

```
template<class I>
concept cpp17-forward-iterator =
  cpp17-input-iterator<I> && Constructible<I> &&
  is_lvalue_reference_v<iter_reference_t<I>> &&
  Same<remove_cvref_t<iter_reference_t<I>>, typename readable_traits<I>::value_type> &&
```

```
  requires(I i) {
    { i++ } -> ConvertibleTo<const I&>;
    { *i++ } -> Same<iter_reference_t<I>>;
  };

template<class I>
concept cpp17-bidirectional-iterator =
  cpp17-forward-iterator<I> && requires(I i) {
    { --i } -> Same<I&>;
    { i-- } -> ConvertibleTo<const I&>;
    { *i-- } -> Same<iter_reference_t<I>>;
  };

template<class I>
concept cpp17-random-access-iterator =
  cpp17-bidirectional-iterator<I> && StrictTotallyOrdered<I> &&
  requires(I i, typename incrementable_traits<I>::difference_type n) {
    { i += n } -> Same<I&>;
    { i -= n } -> Same<I&>;
    { i + n } -> Same<I>;
    { n + i } -> Same<I>;
    { i - n } -> Same<I>;
    { i - i } -> Same<decltype(n)>;
    { i[n] } -> ConvertibleTo<iter_reference_t<I>>;
  };
```

In subclause 24.5.3 [range.subrange], modify in paragraph 1:

```
template<class T>
  concept pair-like = // exposition only
    !is_reference_v<T> && requires(T t) {
      typename tuple_size<T>::type; // ensures tuple_size<T> is complete
      requires DerivedFrom<tuple_size<T>, integral_constant<size_t, 2>>;
      typename tuple_element_t<0, remove_const_t<T>>;
      typename tuple_element_t<1, remove_const_t<T>>;
      { get<0>(t) } -> ConvertibleTo<const tuple_element_t<0, T>&>;
      { get<1>(t) } -> ConvertibleTo<const tuple_element_t<1, T>&>;
    };
```

Furthermore, if additional instances of the *trailing-return-type* production of *return-type-requirement* is approved for application into the Working Draft at the same meeting as this paper is approved (e.g., P1614, “The Mothership Has Landed: Adding <=> to the Library”), replace said production with `-> ConvertibleTo<`*Type*`>` where *Type* is the type named by the *trailing-return-type*:

```
    { /*…*/ } -> ConvertibleTo<Type>;
```

Acknowledgements

The author would like to thank Walter E. Brown, Casey Carter, Paul Preney, David Stone, and any others who have been missed for their feedback on the topic of this paper. The author would also like to thank JF Bastien, chair of the Evolution Working Group Incubator, for the opportunity to present on this topic during the February 2019 session at Kailua-Kona, Hawaii.